

Tensor-Accelerated Eager Multi-Resolution Grids for Evolving Large-Scale Substrates

Romain Claret
romain.claret@unine.ch
University of Neuchâtel
Neuchâtel, Switzerland

Paul Cotofrei
paul.cotofrei@unine.ch
University of Neuchâtel
Neuchâtel, Switzerland

Michael O'Neill
m.oneill@ucd.ie
University College Dublin
Dublin, Ireland

Kilian Stoffel
kilian.stoffel@unine.ch
University of Neuchâtel
Neuchâtel, Switzerland

Abstract

Evolvable-Substrate HyperNEAT (ES-HyperNEAT) discovers neural network topology by querying a Compositional Pattern-Producing Network (CPPN) at spatial positions and placing nodes where output variance is high. Its sequential quadtree cannot be tensorized directly: each network produces a different tree with dynamic shapes. We present Eager Multi-Resolution HyperNEAT (EMR-HyperNEAT), which reformulates adaptive substrate discovery as a batch tensor operation: shared position grids are precomputed for all depths, every position is evaluated in one vectorized CPPN call, then the same variance criterion filters the output. This tensor formulation supports population-level batching, one-time JIT compilation, chunked memory streaming, and six substrate configurations (feedforward through full recurrent) that sequential traversals could not support at scale. On the CRSP/Compustat financial dataset (94K samples), EMR-HyperNEAT achieves 5.5× speedup at depth 6; on XOR (pop 1000), 12–34× per-generation GPU speedup at depths 5–7 (~100× over 30 generations). The reformulation also discovers a superset of ES-HyperNEAT’s positions, yielding higher empirical solve rates. We validate substrate evolution up to depth 13 (358M positions, streamed from disk).

CCS Concepts

• **Computing methodologies** → **Neural networks**; **Genetic algorithms**; **Parallel algorithms**.

Keywords

neuroevolution, ES-HyperNEAT, adaptive substrates, tensor reformulation, indirect encoding, recurrent networks

ACM Reference Format:

Romain Claret, Michael O'Neill, Paul Cotofrei, and Kilian Stoffel. 2026. Tensor-Accelerated Eager Multi-Resolution Grids for Evolving Large-Scale Substrates. In *Genetic and Evolutionary Computation Conference (GECCO Companion '26)*, July 13–17, 2026, San Jose, Costa Rica. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3795101.3805361>



This work is licensed under a Creative Commons Attribution 4.0 International License. *GECCO Companion '26*, San Jose, Costa Rica
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2488-6/2026/07
<https://doi.org/10.1145/3795101.3805361>

1 Introduction

In neuroevolution, indirect encoding generates neural network connectivity from a compact genome rather than specifying each connection. ES-HyperNEAT [5] automatically discovers where to place hidden nodes by examining CPPN [6] output patterns: it recursively subdivides space using a quadtree, expanding regions where CPPN outputs show high variance. This adaptive approach discovers network topology without manual substrate specification, extending the fixed-grid HyperNEAT [7] framework built on NEAT [8].

However, the quadtree resists tensorization. Each depth level depends on the parent’s variance, forcing sequential evaluation. Different CPPNs produce different subdivision patterns, preventing batching. And variable leaf counts are incompatible with JAX’s [1] static shape requirement for JIT compilation. Our prior work confirmed these limits at depths exceeding 5 [3], and a JAX reimplementations of the quadtree yielded only marginal speedup despite batched optimizations, motivating the eager reformulation presented here.

We present EMR-HyperNEAT, which evaluates all positions at all resolutions up front, then filters using the same variance criterion: ES-HyperNEAT’s `subdivide_if(var >  $\theta$ )` becomes `eval_all()`; `filter(var >  $\theta$ )` (Figure 1). This performs more CPPN queries than necessary, but all queries become independent and parallelizable across both cores and population members, reducing complexity from $O(4^D)$ to $O(4^D/P)$ across P parallel cores (Section 3). Recurrent substrate configurations become feasible through a connection type taxonomy (Section 4). Section 5 validates 12–34× on-device GPU speedup on XOR at depths 5–7, and empirically higher solve rates across benchmarks.

2 Background

NEAT [8] evolves neural network topology and weights simultaneously. HyperNEAT [7] extends NEAT with indirect encoding: a CPPN maps spatial coordinates (x_1, y_1, x_2, y_2) to connection weights, so compact genomes can specify large-scale networks. ES-HyperNEAT [5] removes the need for a predefined substrate by adaptively placing nodes via quadtree subdivision. In Phase 1, for each input coordinate, the quadtree expands regions where CPPN variance exceeds threshold θ_{div} , discovering H hidden positions. In Phase 2, each of the H positions undergoes its own quadtree traversal to discover hidden-to-hidden connections, requiring $O(H^2)$

sequential queries. Since cost grows as 4^D , each additional depth roughly quadruples generation time.

The quadtree traversal resists tensor batching for three reasons: (1) non-uniform subdivision patterns prevent batching via `vmap`; (2) children cannot be evaluated until parent variance is computed; and (3) variable leaf counts are incompatible with JAX’s static shape requirement for JIT compilation.

3 EMR-HyperNEAT Algorithm

EMR-HyperNEAT replaces ES-HyperNEAT’s sequential quadtree with three parallelizable stages: (1) precompute static multiresolution grids, (2) batch-evaluate all CPPN queries via `vmap`, and (3) apply hierarchical variance filtering.

3.1 Hierarchical Grid and Batch Query

At initialization, EMR-HyperNEAT precomputes a complete multiresolution grid from depth 0 to maximum depth D :

$$\mathcal{G}_D = \bigcup_{d=0}^D \text{Grid}_d, \text{Grid}_d = \left\{ \left(\frac{2i+1}{2^{d+1}} - 1, \frac{2j+1}{2^{d+1}} - 1 \right) : i, j \in [0, 2^{d+1}) \right\} \quad (1)$$

The total position count is $N_{\text{total}} = (4^{D+2} - 4)/3$, growing from 1,364 at depth 4 to 87,380 at depth 7 and 358M at depth 13. All arrays have static shapes, compatible with JIT compilation. With positions in static arrays, CPPN queries become independent and parallel via $\mathbf{W} = \text{vmap}(\text{CPPN})(s, T)$, replacing N sequential queries with $O(N/P)$ wall-clock time. Population parallelization evaluates all genomes simultaneously.

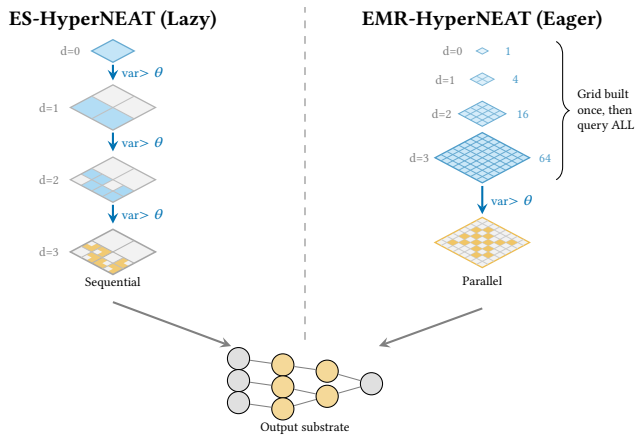


Figure 1: Lazy vs eager substrate discovery. Left: ES-HyperNEAT subdivides sequentially. Right: EMR-HyperNEAT evaluates all depths in parallel, then filters by variance.

3.2 Variance Filtering and Active Mask

Variance is computed bottom-up from children to parents. The active mask propagates top-down:

$$\text{active}[d][i] = \begin{cases} \text{True} & d = 0 \\ \text{active}[d-1][p_i] \wedge (\text{var}[d-1][p_i] > \theta_{\text{div}}) & \text{else} \end{cases} \quad (2)$$

Each parent controls all four children as a block, so filtering runs in parallel. Table 1 summarizes the complexity comparison.

Table 1: Computational complexity. D = max depth, H = discovered hidden positions, P = parallel execution units.

Operation	ES-HyperNEAT	EMR-HyperNEAT
Grid Construction	—	$O(4^D)$ once
CPPN Query (Phase 1)	$O(4^D)$ seq.	$O(4^D/P)$
Variance Comp.	$O(4^D)$ seq.	$O(4^D/P)$
Mask Propagation	$O(4^D)$ seq.	$O(4^D/P)$
h→h Query (Phase 2)	$O(H^2)$ seq.	$O(H^2/P)$
Total	$O(4^D + H^2)$	$O((4^D + H^2)/P)$

THEOREM 1 (SUPERSET DISCOVERY). *Under identical variance thresholds θ_{div} and band thresholds θ_{band} , EMR-HyperNEAT discovers a superset of the positions discovered by ES-HyperNEAT. When both algorithms discover identical position sets, they produce identical connection sets.*

PROOF. ES-HyperNEAT’s quadtree prunes subtrees when parent variance falls below θ_{div} . Since low parent variance does not imply low child variance, high-variance children within low-variance parents are never evaluated. EMR-HyperNEAT’s eager evaluation computes all positions before applying variance masks, recovering these positions. Therefore every position discovered by ES-HyperNEAT is also discovered by EMR-HyperNEAT (superset property). This also handles `initial_depth=0` robustly: ES-HyperNEAT may discover no nodes if root variance is insufficient, while EMR-HyperNEAT evaluates all positions regardless. $\square$

3.3 Threshold Adaptation

The two approaches compute variance over different regions: ES-HyperNEAT only looks inside regions whose parent already passed the threshold, while EMR-HyperNEAT looks at all positions including low-variance regions. ES-HyperNEAT’s original thresholds ($\theta_{\text{div}}=0.5$, $\theta_{\text{band}}=0.3$) produce no solutions under eager evaluation. EMR-HyperNEAT uses adapted thresholds ($\theta_{\text{var}}=0.03$, weight magnitude filtering at 0.1) that are functionally equivalent for the eager context. This adaptation is a necessary consequence of the reformulation, not an independent parameter choice.

4 Connection Type Taxonomy

EMR-HyperNEAT’s parallelization of Phase 2 makes recurrent substrate configurations computationally feasible. In HyperNEAT substrates, comparing source and target Y coordinates classifies connections into four primitives: **Forward** ($y_s < y_t$), **Backward** ($y_s > y_t$, feedback), **Lateral** ($y_s = y_t$, same layer), and **Self-loop** ($s = t$).

Enabling or disabling these produces six substrate configurations (Table 2).

Table 2: Six substrate configurations available in EMR-HyperNEAT. F=Forward, H=hidden-to-hidden, B=Backward, L=Lateral, S=Self.

Config	F	H	B	L	S	Notes
Feedforward	✓					Fastest; skips Phase 2
h→h	✓	✓				ES-HN default
Backward	✓	✓	✓			Feedback loops
Lateral	✓	✓		✓		Same-layer links
Self	✓	✓			✓	State memory
Full Recurrent	✓	✓	✓	✓	✓	All connections

Cross-problem evaluation (depth 5, pop 300, 10 replications) shows connection types significantly affect convergence (Table 3). On XOR, Backward and Lateral achieve 100% ($p < 0.001$ vs Feedforward’s 20%). On Visual Discrimination (2D spatial classification) [7], Backward achieves 100% convergence while Lateral and Self hurt performance.

Table 3: Cross-problem convergence rate (%) by connection type. Bold = best or tied. p : Fisher’s exact, best vs feedforward.

Problem	FF	h→h	Back	Lat	Self	Full	p
XOR	20	50	100	100	90	60	<0.001
Visual	90	70	100	40	40	90	<0.001
Retina	100	80	90	80	100	100	n.s.

5 Experimental Evaluation

5.1 Setup

CPU: Apple M4 Max (JAX CPU backend). GPU: NVIDIA RTX 2080 Ti 11GB (JAX CUDA backend). Baseline: PUREPLES [4], the reference Python implementation of ES-HyperNEAT (ES-HN). Problem: XOR (4 input patterns), standard neuroevolution benchmark [8]. Both algorithms: 30 generations, 8 populations (50–1000), depths 1–7, h→h connection type, 3 replications (504 runs total). EMR-HyperNEAT implemented in JAX/Python on TensorNEAT [9]; each experiment ran in process isolation to ensure timing includes JIT compilation overhead.

5.2 Scaling and Speedup

Table 4 presents per-generation timing. Figure 2 shows total runtime scaling including depths 8–13 via memory streaming, with cumulative runtime IQR median approaching ~100× over 30 generations at depth 7.

Statistical validation (Table 5) confirms that at depths ≥ 5 , EMR-HyperNEAT GPU achieves significant speedups ($p < .001$, Cliff’s $\delta \geq 0.99$, large effect) after Holm-Bonferroni correction. Speedup increases with depth (Spearman $\rho = 0.96$, $p < .001$). Runtime variance also diverges: at depth 7, estimated σ is 3.6h (ES-HN), 31min (EMR CPU), 1.9min (EMR GPU), a 114× tighter bound.

Table 4: Average time per generation (XOR, pop=1000, h→h type). Speedup excludes one-time JIT compilation.

Depth	ES-HN	EMR	JIT	Speedup CPU/GPU
1–3	0.3–1.0s	1–3s / 0.9–2.3s	4–8s / 7–17s	0.2–0.3×
4	4.27s	5s / 2.9s	12s / 18s	0.85× / 1.5×
5	46.3s	10s / 3.9s	15s / 19s	4.6× / 12×
6	200.4s	28s / 8.9s	30s / 24s	7.2× / 23×
7	2,020s	165s / 60s [†]	60s / 39s	12.2× / 34× [†]

[†]GPU depth 7 uses memory streaming (50–70s/gen on 11GB VRAM).

Table 5: Statistical comparison vs ES-HN: Mann-Whitney U (one-sided), Cliff’s delta. Significant at $\alpha = 0.05$ for GPU depths ≥ 5 after Holm-Bonferroni correction.

Depth	EMR GPU			EMR CPU		
	p	δ	Speedup	p	δ	Speedup
1–3	>0.99	−1.00 ^L	0.1–0.3×	>0.99	−1.00 ^L	0.1–0.3×
4	<.001	0.59 ^L	1.6×	.915	−0.23 ^S	0.8×
5	<.001	0.99 ^L	6.8×	<.001	0.70 ^L	3.5×
6	<.001	1.00 ^L	42×	<.001	0.98 ^L	11×
7	<.001	1.00 ^L	75×	<.001	0.91 ^L	15×

Effect size: S=small, L=large.

5.3 Solution Quality

Table 6 shows EMR-HyperNEAT achieves equal or higher solve rates. At depth 6, ES-HyperNEAT solves XOR in only 33% of runs (pop=300) while EMR-HyperNEAT achieves 100%, due to bottom-up evaluation recovering positions that top-down traversal misses. Both achieve identical maximum fitness (0.99+) when successful.

Table 6: Per-generation time and solve rate (XOR, h→h type with cache). JIT = one-time compilation overhead.

Depth	Algorithm	Pop 300	+ JIT	Pop 500	+ JIT
4	ES-HN	1.5s / 67%	-	2.5s / 33%	-
	EMR (CPU)	2s / 100%	11s	3s / 100%	10s
	EMR (GPU)	0.9s / 100%	10s	1.3s / 100%	12s
5	ES-HN	6.8s / 0%	-	22.2s / 100%	-
	EMR (CPU)	6s / 100%	15s	7s / 100%	14s
	EMR (GPU)	1.2s / 100%	10s	1.9s / 100%	13s
6	ES-HN	147.8s / 33%	-	220.5s / 0%	-
	EMR (CPU)	12s / 100%	16s	40s / 100%	31s
	EMR (GPU)	2.6s / 100%	12s	4.3s / 100%	15s
7	ES-HN	652s / 0%	-	1,649s / 0%	-
	EMR (CPU)	48s / 100%	46s	94s / 100%	72s
	EMR (GPU)	7.9s / 100%	17s	13s / 100%	23s

5.4 Real-World Benchmark

Table 7 compares performance on CRSP/Compustat Merged financial data (94,464 observations) [2]. At depth 6, EMR-HyperNEAT is 5.5× faster despite streaming deeper layers from CPU RAM when GPU memory is insufficient. ES-HyperNEAT exhibits exponential

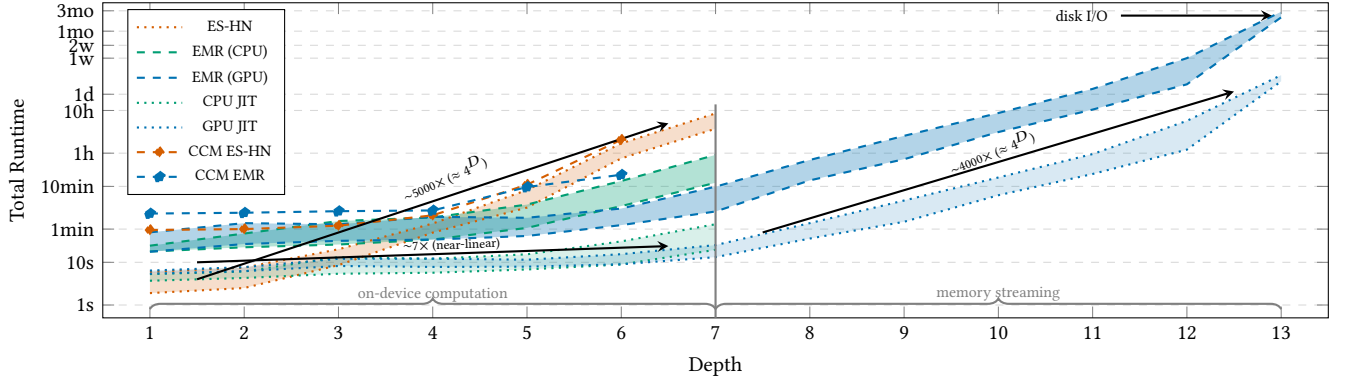


Figure 2: Runtime over 30 generations (log y-axis). IQR bands: d1–7 use 8 pops $\times$ 3 seeds; d8–13 use GPU $\leftrightarrow$ RAM streaming (pop 300, 1000, 3 seeds). The d13 widening reflects disk I/O dominating JIT and per-gen cost. CCM lines: financial data [2] (94K samples, pop 100). Trend arrows: $\sim 5000\times$ ES-HN ($\approx 4^D$), $\sim 7\times$ EMR on-device, $\sim 4000\times$ EMR streaming ($\approx 4^D$).

Table 7: Per-generation time on CCM financial dataset (94,464 samples, pop=100). EMR uses h $\rightarrow$ h type (GPU).

Depth	ES-HN	EMR	EMR JIT	Speedup
1–3	1.9–2.4s	5.5–6.2s	11–13s	0.35–0.39 $\times$
4	4.2s	6.6s	13s	0.64 $\times$
5	21.9s	23s	46s	0.95 $\times$
6	247.9s	45s [†]	89s	5.5 $\times$

[†]Streams deeper layers from CPU RAM (GPU memory insufficient).

scaling (depth 5 $\rightarrow$ 6: 11.3 $\times$ slower); EMR-HyperNEAT scales linearly on-device.

6 Discussion

The threshold adaptation (0.03 vs 0.5) is a necessary consequence of the reformulation: ES-HyperNEAT’s original thresholds produce no solutions under eager evaluation because variance is computed over all positions rather than only inside already-accepted regions. The adapted thresholds produce more hidden node candidates, contributing to the higher solve rates at depths 5–7 (Table 6), at the cost of redundant CPPN queries amortized by parallelism. EMR-HyperNEAT is advantageous when JIT overhead (4–60s) is amortized across generations. JAX caches compiled functions, so experiments at matching depth and population size skip JIT, and the precomputed grid (~ 683 KB at depth 7) is reused across problems. At low depth with few generations, ES-HyperNEAT remains preferable.

Caching accelerates computation further: cached h $\rightarrow$ h connections reduce subsequent discovery from ~ 27 s to ~ 3 ms. Memory scales as 4^D : depth 5 < 620 MB, depth 7 ~ 10 GB (pop 500). At depth 13 (358M positions, pop 300), weight arrays stored on disk dominate both JIT (~ 5.3 h) and per-gen cost (~ 5.8 h), visible as the d12 $\rightarrow$ d13 widening in Figure 2.

7 Conclusion

To our knowledge, EMR-HyperNEAT is the first massively parallel adaptive substrate discovery algorithm. The lazy-to-eager reformulation yields significant speedups that grow with depth (Tables 4–7),

validated on both XOR and financial data (94K samples). Beyond speedup, bottom-up evaluation discovers a superset of substrate positions (Theorem 1), which yields higher solve rates in practice, and the connection type taxonomy makes recurrent substrates computationally tractable. This makes previously unreachable substrate configurations accessible, opening per-node substrate complexification at new depth and population scales, though further efficiency gains via selective evaluation remain constrained by the need to preserve parallelization.

Code Availability

<https://github.com/RomainClaret/emr-hyperneat>

Acknowledgments

Generative AI tools assisted with visualizations, pseudocode conversion, and draft revision.

References

- [1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. 2018. JAX: composable transformations of Python+NumPy programs. (2018).
- [2] Center for Research in Security Prices (CRSP). 2019. *CRSP/Compustat Merged Database (CCM)*. CRSP. <https://www.crsp.org/products/research-products/crspcompustat-merged-database>. Accessed via WRDS.
- [3] Romain Claret, Michael O’Neill, Paul Cotofrei, and Kilian Stoffel. 2024. Investigating Hyperparameter Optimization and Transferability for ES-HyperNEAT: A TPE Approach. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 1879–1887.
- [4] Westh et al. 2017. *Pureples - pure python library for es-hyperneat*.
- [5] Sebastian Risi and Kenneth O Stanley. 2012. An enhanced hypercube-based encoding for evolving the placement, density, and connectivity of neurons. *Artificial life* 18, 4 (2012), 331–363.
- [6] Kenneth O Stanley. 2007. Compositional pattern producing networks: A novel abstraction of development. *Genetic programming and evolvable machines* 8, 2 (2007), 131–162.
- [7] Kenneth O Stanley, David B D’Ambrosio, and Jason Gauci. 2009. A hypercube-based encoding for evolving large-scale neural networks. *Artificial life* 15, 2 (2009), 185–212.
- [8] Kenneth O Stanley and Risto Miikkulainen. 2002. Evolving neural networks through augmenting topologies. *Evolutionary computation* 10, 2 (2002), 99–127.
- [9] Lishuang Wang, Mengfei Zhao, Enyu Liu, Kebin Sun, and Ran Cheng. 2024. Tensorized NeuroEvolution of augmenting topologies for GPU acceleration. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1156–1164.

Received 19 January 2026; revised 10 March 2026; accepted 27 January 2026